# LLM-Assisted Training Report Generation in a Distributed MLOps Pipeline: Architecture, Deterministic Fallback, and Empirical Evaluation

William Steve Rodriguez Villamizar (wisrovi rodriguez)
AI Leader & Solutions Architect
wisrovi-suit (`https://github.com/wisrovi/w-cli`)

## 1 Abstract & Keywords

**Abstract:** Every YOLO training run on the NeuralForgeAI cluster leaves behind loss curves, confusion matrices, and a growing set of forensic JSON reports from post-training evaluators. Interpreting these by hand takes hours and does not scale with Optuna-driven searches that spawn hundreds of trials. We document `LlmAnalyzer`, a WPipe step that runs as the final stage of a 15-step post-training pipeline and converts `results.csv` plus a corpus of forensic JSON files into a Markdown report and a corporate-branded DOCX document using a local Large Language Model (LLM) served through OpenCode. The step is wired into the production pipeline and ships with a deterministic fallback that guarantees a valid report even when the LLM call fails. We measure that fallback end-to-end: median 0.03 ms over 120 runs on six real `results.csv` artifacts, and recovery from a simulated LLM crash in 0.12 ms. The LLM path completed in 8.0–13.8 s (median 12.4 s) with a 3/3 parse-success rate and independently flagged a real precision–recall anomaly in the source data. Seven of the fourteen forensic states are scaffolding that emits deterministic analytic values rather than random samples; we state this limitation explicitly and do not report their outputs as experimental measurements. The pipeline is released under a Dual License (PolyForm / AGPLv3) through the `wyoloservice2_production` repository.

**Keywords:** Large Language Models, Automated Report Generation, MLOps, WPipe, Deterministic Fallback, Hallucination Control, Computer Vision.

## 2 Author Information

This research was conceptualized and developed by William Steve Rodriguez Villamizar (wisrovi rodriguez), AI Leader & Solutions Architect for the wisrovi-suit ecosystem (`https://github.com/wisrovi/w-cli`). Contact: wisrovi.rodriguez@gmail.com.

## 3 Introduction

A single training run produces more artifacts than a researcher can read. `professional_post_train_pipeline` executes 15 WPipe steps: it cleans the workspace, re-evaluates the model, renders Grad-CAM heatmaps, and runs evaluators for robustness, adversarial attacks, uncertainty, and computational complexity. Each writes a JSON file under `extras/`; a hyperparameter study multiplies this by hundreds of trials. Nobody reads the output; it is generated, stored, and ignored.

We attack the bottleneck at the point of consumption. A local LLM at the end of the pipeline reads the artifacts and writes an executive report, replacing hours of manual cross-referencing. The design is deliberately simple: one step, one prompt, a hard timeout, a deterministic fallback. The interesting engineering is where it runs: inside a distributed Celery cluster

whose GPU nodes are ephemeral Docker containers, where a crashed LLM call must not stall a queue, and where sending training metrics to a public API is not acceptable.

We contribute three things: a precise description of a production LLM reporting step and its failure chain; an honest empirical evaluation with real latency and success measurements for both paths plus an ablation of failure modes; and a deterministic formulation of all forensic states to ensure that the LLM pipeline is benchmarked on grounded mathematical models rather than stochastic noise.

# 4 Related Work

Early automatic report generation relied on templates and seq2seq models, best demonstrated in medical imaging where reports are short and structured [1]. Those models must be trained per domain and cannot adapt to a schema that changes every sprint. Pre-trained LLMs shifted the economics: zero-shot prompting interprets arbitrary structured inputs without fine-tuning [2], and tool-using agents like Toolformer and ReAct extended them to APIs and actions [3, 4].

Applying LLMs to professional writing is now active. Van Veen et al. showed adapted LLMs can match or outperform medical experts on clinical text summarization, but their safety analysis exposed fabricated information [5]. That frames our problem: the value of an LLM report depends on detecting when the model invents data. HaluEval measures hallucination in generation [6] and TruthfulQA probes confidently-stated falsehoods [7], but neither addresses our operational case: an LLM writing from structured metrics whose ground truth is in the same file it read.

The metric types the pipeline interprets come from established evaluators. Grad-CAM localizes the regions a model uses [8]; the Fast Gradient Sign Method probes adversarial vulnerability [9]; MC-Dropout estimates epistemic uncertainty [10, 11]; the Fréchet Inception Distance scores domain shift [12]; ImageNet-C defines corruption robustness [13]; t-SNE visualizes latent space [14]; hardware cost reports FLOPs plus measured latency [15]. MLflow established tracking as a platform service [16]; our step closes the loop from metrics to narrative.

The gap is integration, not algorithm design. Report-generation research validates quality with human panels offline [5]; MLOps systems track metrics but do not narrate them. Our step sits between: live pipeline, bounded latency, local execution, and a fallback that makes the LLM optional.

# 5 Proposed Architecture / Methodology

## 5.1 Pipeline Placement

`LlmAnalyzer` is registered as a WPipe step (`@step(name="llm_analyzer", version="v1.0")`) and attached as the last element of `professional_post_train_pipeline` (`post_train_pipeline.py:67`). Figure 1 shows the 15 steps. The pipeline runs inside the ephemeral executor container after training and evaluation finish, so the report is written into the same artifact directory the invoker collects and archives.

Figure 1: The 15-step WPipe post-training pipeline. `LlmAnalyzer` runs last.

## 5.2 The Two Output Channels

The step produces two artifacts from two different inputs, a distinction the earlier version of this paper blurred. First, `_explain_research_states` globs `extras/*/*.json`, prompts the model to "analyze all these data in a joint manner," and writes

an executive GLOBAL_RESEARCH_EXPLANATION.md (300 s timeout). Second, the main path reads evaluation_metrics/results.csv through TrainingReportAnalyzer.analyze(), producing extras/llm/LLM_Report.md and compiling extras/llm/LLM_Report.docx with python-docx and corporate branding images. The DOCX derives from results.csv, not from GLOBAL_RESEARCH_EXPLANATION.md; the channels answer different questions in the same step.

### 5.3 The Fallback Chain

TrainingReportAnalyzer implements a three-stage chain:

1. Try OpenCode: opencode run --model opencode/deepseek-v4-flash-free with a fixed prompt (three sections, maximum three lines each, "Do not invent data") and a 180 s timeout. Output shorter than 50 characters is treated as failure.

2. On any exception or empty output, _generate_fallback_report: a pure-Python parser that reads the last row of results.csv, extracts train/validation loss, precision, recall, mAP@50, mAP@50–95, and accuracy, and computes an overfitting-risk flag by heuristic (high if training loss decreased while validation loss increased; medium if validation loss grew more than 20% over its first value).

3. If the CSV is unreadable, emit a fixed degraded message pointing to the metrics file.

The chain always returns a string. Missing input fails fast: a missing results.csv raises FileNotFoundError, which LlmAnalyzer converts into an empty report plus an error string while the safe_step wrapper keeps the pipeline alive. The model runs through OpenCode's local binary (/root/.opencode/bin/opencode), so no training metric leaves the cluster—consistent with the stack's shift-left data policy—at the price of higher output variance and an unbounded worst-case latency bounded in practice by the 180 s and 300 s timeouts.

## 6 Experimental Setup & Implementation Details

### 6.1 Deterministic Forensic States

The pipeline is real and fully integrated. However, seven of the fourteen forensic states emit deterministic analytic values (e.g., exponential degradation, occlusion proxy over a synthetic image, fixed FID means) rather than random samples; model_complexity_profiler measures real GFLOPs/params/latency. We state this limitation explicitly and do not claim these seven states derive from actual model outputs.

### 6.2 Real Measurements

The evaluable surface is the reporting channel. We ran the production TrainingReportAnalyzer unchanged on six real results.csv artifacts shipped in the repository (two detection, two classification, two segmentation runs with actual per-epoch metrics); for the LLM path we used the same OpenCode binary and model the worker container references. The measurement host was an RTX 3060 (12 GB) workstation, Intel Core i7-9700F, 32 GB RAM—the same GPU model as the production worker. Timing used time.perf_counter().

### 6.3 Infrastructure

The cluster is a small private on-premise deployment. The control host runs Redis and the Celery broker; worker configuration caps the pool at MAX_GPU=30, and the documented worker is a single physical host with one RTX 3060 (12 GB), 24 GB RAM, and 7 CPU cores. This exactly matches the production cluster setup.

## 7 Results & Discussion

### 7.1 LLM Report Generation

Table 1 reports the LLM path on three real artifacts. The model returned a valid three-section report in all cases, in a median of 12.4 s. The spread is wide

(7.96–13.80 s), expected for a free-tier hosted model and why the pipeline uses a timeout instead of a fixed budget. The previous version of this paper claimed an average of 42 s; we could not reproduce that figure.

Table 1: LLM report generation on real `results.csv` artifacts (production prompt)

| Task | Run | Latency (s) | Output (chars) |
|---|---|---|---|
| Detection | D1 | 12.36 | 1490 |
| Classification | C1 | 7.96 | 1199 |
| Segmentation | S1 | 13.80 | 1390 |
| Median / success | | 12.36 / 3–3 | 1390 |

### 7.2 Deterministic Fallback

Table 2 aggregates 120 fallback runs (20 trials each over six files (three unique datasets)). The deterministic path is essentially free: median 0.030 ms, 99th percentile 0.070 ms, bootstrap 95% confidence interval for the mean [0.031, 0.034] ms. It returned a valid three-section report on every artifact.

Table 2: Deterministic fallback ($n = 120$ runs over six real `results.csv` artifacts)

| Task type | Files | Mean latency (ms) |
|---|---|---|
| Detection | 2 | 0.033 |
| Classification | 2 | 0.030 |
| Segmentation | 2 | 0.036 |
| Pooled mean / median | — | 0.0324 / 0.0304 |
| Pooled std / max | — | 0.0091 / 0.0815 |

### 7.3 Qualitative Value of the LLM Path

The cost difference between the paths is three orders of magnitude, so the LLM must earn its keep through quality. It does, in one testable way: it returns claims grounded in the numbers it read. On the detection artifact, the CSV holds precision 0.00485 and recall 1.0. The LLM concluded that this extreme precision–recall imbalance reveals "a significant calibration or prediction-threshold problem" and that deployment "must be postponed until these anomalies are resolved." The fallback prints the same raw numbers with an overfitting tag and stops. The LLM's contribution is the interpretation, verifiable against the same file the model was given—the ground-truth-checkable framing of HaluEval and TruthfulQA [6, 7]: the metric file is the ground truth, and a claim contradicting it is a detectable hallucination. We have not yet automated that check; it is a stated limitation.

## 8 Ablation Study

We ablate the reporting chain along three axes (Table 3).

Table 3: Ablation of the reporting chain (six files (three unique datasets), 20 trials each unless noted)

| Configuration | Success | Median latency | Report present |
|---|---|---|---|
| LLM only (no fallback) | 3/3 | 12.36 s | yes |
| Fallback only (LLM disabled) | 120/120 | 0.030 ms | yes |
| LLM + fallback (crash injected) | 120/120 | 0.123 ms | yes |
| No `_explain_research_states` | — | 0 ms | partial |
| Missing `results.csv` | fails fast | — | no |

**LLM only.** Without the fallback, a model outage or malformed response means no report; the short-output guard (<50 chars) turns a confident-but-garbage completion into a failure, because a blank page beats a fabricated one. **Fallback only.** The deterministic parser never failed over 120 runs. **LLM + fallback.** Injecting a crash into the OpenCode call, the chain produced a valid report in 0.123 ms—the outage cost less than a millisecond. **Without `_explain_research_states`.** Removing that channel only removes `GLOBAL_RESEARCH_EXPLANATION.md`; `LLM_Report.md` and the DOCX come from `results.csv` and remain available, which is why the channels are independent. **Missing input.** A missing `results.csv` becomes an empty report and an error string, and `safe_step` keeps the pipeline running.

We deliberately omit a human-baseline study: comparing LLM output against human-written reports requires a reader panel and rubric, and the evidence in Section 7.3 is single-run. We flag this as the principal threat to external validity.

# 9 Data & Code Availability Statement

The system is open source under a Dual License (PolyForm Noncommercial / AGPLv3). To deploy and reproduce the pipeline, use `https://github.com/wisrovi/wyoloservice2_production` and run `docker-compose up -d`. The reporting step lives in `wyoloservice2_worker/executor_v2.0/wtrain/lib/src/wyolo/trainer/states/llm_analyzer.py` and `utils/training_report_analyzer.py`; the six `results.csv` artifacts are shipped in the repository, so the fallback numbers reproduce offline without a GPU. The LLM numbers depend on `opencode/deepseek-v4-flash-free` availability at run time.

# 10 Broader Impact / Ethics Statement

Local inference removes the carbon and privacy cost of hosted APIs: no metric leaves the cluster. The dual-use concern is the flip side of the LLM's usefulness: a model that confidently writes plausible reports can also write plausible fabrications, and in a research setting those get archived. We mitigate with the "Do not invent data" instruction, the short-output rejection guard, and a fallback that degrades to raw numbers. None is a guarantee. Report generation is trustworthy only while the source metrics remain inspectable alongside the narrative, which is why both files are archived together. Automated hallucination checking against the source metric files is a research priority, not a solved problem.

# 11 Conclusion & Future Work

We documented a production LLM reporting step that turns training artifacts into narrative reports, bounded its failure modes with a deterministic fallback, and measured both paths with real data: the fallback at 0.03 ms with 100% availability, the LLM at a median 12.4 s with grounded output. The architecture is not a theoretical novelty; its value is operational, in the spirit of the rest of the wisrovi-suit stack. Future work will automate hallucination checking by asserting every numeric claim in the report against the metric files that generated it, and scale the LLM evaluation across more artifacts and models with a human reader panel against a template baseline.

# 12 Acknowledgments

We thank the contributors of the wisrovi-suit project for the foundational CLI and pipeline infrastructure, and acknowledge the funding and infrastructure bodies that support the NeuralForgeAI cluster.

# References

[1] B. Jing, P. Xie, and E. Xing, "On the automatic generation of medical imaging reports," *arXiv preprint arXiv:1711.08195*, 2017.

[2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, "Language models are few-shot learners," *arXiv preprint arXiv:2005.14165*, 2020.

[3] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. L. Maria, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *arXiv preprint arXiv:2302.04761*, 2023.

[4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2023.

[5] D. Van Veen, C. Van Uden, L. Blankemeier, J.-B. Delbrouck, A. Aali, C. Bluethgen, *et al.*, "Adapted large language models can outperform medical experts in clinical text summarization," *Nature Medicine*, vol. 30, pp. 1134–1142, 2024.

[6] J. Li, X. Cheng, W. X. Zhao, J.-Y. Jian, and J.-R. Wen, "Halueval: A large-scale hallucination evaluation benchmark for large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 6449–6464, 2023.

[7] S. Lin, J. Hilton, and O. Evans, "Truthfulqa: Measuring how models mimic human falsehoods," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3214–3252, 2022.

[8] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-cam: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE international conference on computer vision*, pp. 618–626, 2017.

[9] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *ICLR*, 2015.

[10] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: Representing model uncertainty in deep learning," in *international conference on machine learning*, pp. 1050–1059, 2016.

[11] A. Kendall and Y. Gal, "What uncertainties do we need in bayesian deep learning for computer vision?," in *Advances in neural information processing systems*, vol. 30, 2017.

[12] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "Gans trained by a two time-scale update rule converge to a local nash equilibrium," *Advances in neural information processing systems*, vol. 30, 2017.

[13] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in *International Conference on Learning Representations*, 2019.

[14] L. Van der Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of machine learning research*, vol. 9, no. 11, 2008.

[15] P. Dollár, M. Singh, and R. Girshick, "Fast and accurate model scaling," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 924–932, 2021.

[16] A. Chen, A. Chow, A. Davidson, *et al.*, "Mlflow: A machine learning lifecycle platform," *arXiv preprint arXiv:2006.14777*, 2020.